

Take-Home Assignment: Meteora DLMM Swap Indexer

Overview

Build a small service that indexes swap activity from Meteora's DLMM program on Solana mainnet, stores it in a database, and serves hourly and daily volume over a REST API. We expect roughly 15–20 hours of focused work.

We care more about correctness, idempotency and clear reasoning than feature count. A small system that is correct beats a big one that is fragile.

Background

- **DLMM program ID:** `LBUZKhRxPF3XUpBCjp4YzTKgLccjZhTSDM9Pd8ZqK3m`
- **IDL and reference SDK:** public @ <https://github.com/MeteoraAg/dlmm-sdk>
- **Pools to index:** swaps from **all** DLMM pools.
- **RPC:** any free mainnet RPC works (Helius and Triton free tiers are fine). A free tier may not keep up with the program's full transaction volume. If yours can't, tell us how you handled it and what you would do with a paid tier.
- **Price feed:** use any price source you like to convert volume to USD. The Jupiter Price API, the Birdeye API and Binance kline (candlestick) data are all fine, and so is anything similar. Explain your choice in the write-up.

Requirements

1. Ingestion

- **Data source:** index new DLMM swaps as they land on chain. Choose your source and explain the choice in your write-up. Any of these is acceptable:
 - **RPC polling:** for example `getSignaturesForAddress` followed by `getTransaction`.
 - **Websocket subscriptions:** for example `logsSubscribe` or `programSubscribe`.

- **Geyser:** a Yellowstone gRPC stream, if you have access to one. Several providers offer trials.
- **Resilience:** your indexer must not silently lose swaps when its data source fails, for example when a websocket disconnects, a stream stalls, or the RPC is down. Detect the gap and fill it.
- **Restart safety:** if the process is killed and restarted, it must resume from where it stopped, with no gaps and no duplicates.
- **Rate limits:** handle RPC rate limits and transient failures with retries and backoff.

2. Decoding

Extract every DLMM swap, including:

- swaps called directly, and swaps called through another program such as a Jupiter route

A single transaction can contain several swaps, and each one must be stored separately. Skip failed transactions. Take amounts and fees from the swap event the program emits, not from token balance changes.

Store at least: signature, slot, block time, pool, user, amount in, amount out, input mint, fee, and enough information to uniquely identify each swap within its transaction.

3. Storage

- **Database:** use any database you like. Explain why you chose it in your write-up.
- **Schema:** design it yourself, including keys, indexes and column types.
- **Idempotency:** processing the same transaction twice must not change query results.
- **Local setup:** a `docker-compose.yml` must bring up your database and your service.

4. API

Serve a REST API over HTTP. Rust with Axum is preferred.

- **What it must do:** return swap volume for a given pool, bucketed by hour and by day, over a time range the caller chooses. Report volume in both

tokens and in USD.

- **Your design:** endpoint paths, parameters and the response shape are up to you. In your write-up, explain the choices you made and why.

5. Tests

- **Decoder tests:** use real mainnet transactions saved as JSON fixtures from `getTransaction`. Cover the cases listed under Decoding.
- **Idempotency test:** at least one test against your real database that proves ingesting the same data twice changes nothing.

6. Write-up (`DESIGN.md` , 2 pages max)

Explain your design choices and the trade-offs behind them. Cover at least:

1. **Data source:** why you chose polling, websockets or Geyser, and what you gave up by not choosing the others.
2. **Redundancy:** what happens when your websocket connection or stream is interrupted, how you notice, and how you recover the swaps you missed. Would you run more than one ingestion source or instance? If so, how do you keep them from creating duplicates?
3. **Database and schema:** why you chose this database, how you structured the data, and how that structure serves the volume query.
4. **Idempotency:** what uniquely identifies a swap, and how you prevent duplicates.
5. **Decoding:** how you find DLMM swaps inside calls from other programs, and how you decode the swap event.
6. **API design:** the endpoints, parameters and response shape you chose, and why. Include how you handle units and token decimals, time zones and bucket boundaries, and empty buckets.
7. **Pricing:** which price source you used and why. Also cover what you do when a token has no price available, and how you avoid losing precision in the calculation.
8. **Scaling:** where your current design breaks down as volume grows, and what you would replace first.
9. **Next steps:** what you would do with another week.

Extras (optional)

- **Backfill mode:** given a start time, fetch and index historical DLMM swaps, either for one pool or for all of them. It must be resumable: if it's killed partway through, a restart continues where it stopped. It must also run safely alongside live indexing without creating duplicates.

Constraints

- **Language:** Rust for the indexer and the API.
- **AI tools:** allowed. We'll ask you to walk through and extend your code live, so make sure you understand every line.
- **Submission:** a Git repository (private, with access shared to us) containing a README. The README must let us run `docker compose up`, index live swaps, and query the API in under 10 minutes.